Software Engineering Salary Analysis &

Prediction in the Egyptian Market

Supervised by: Dr. Heba El-Hadidy

Team Members

1. Asmaa Ahmed Ismail Maryah	811634546	IS
2. Rofayda Mohamed El-Desouki	811649191	IS
3. Nesma Khairy El-Ghalban	811668583	IS
4. Esraa Saber Ahmed Hafez	812555812	IS
5. Fathy Hassanin Fathy	808971321	IS

Table of Contents

1.Technical Abstract:

This section outlines the methodology used to transform raw, heterogeneous salary data into a structured dataset suitable for predictive modeling. The primary focus was on data integrity, categorical standardization, and outlier mitigation.

1.1 Data Integration and Structural Cleanup

The initial phase involved merging disparate datasets to create a unified data frame. Redundant features, such as `Timestamp` and `Company Name`, were dropped to ensure privacy and focus purely on economic variables. String normalization was applied across all categorical columns (e.g., converting to lowercase and stripping whitespace) to prevent duplication caused by case sensitivity.

1.2 Feature Engineering and Standardization

- **Experience Quantization:** Raw experience strings (e.g., "3-5 years") were parsed to extract numerical values. A logic-based mapping was implemented to convert qualitative ranges into a consistent "Years of Experience" integer format.
- **Role Categorization:** Using string-matching algorithms, fragmented job titles were consolidated into primary industry pillars (e.g., "Backend," "Frontend," "Data Science," and "Mobile Development"). This reduced high-cardinality noise and improved the statistical significance of each category.
- **Currency & Salary Normalization:** All salary entries were filtered to ensure consistent currency units. Entries lacking clear salary figures or using non-standard payment intervals were excluded to maintain the target variable's reliability.

1.3 Data Quality Assurance (Outlier Detection)

To protect the model from skewed results, the Interquartile Range (IQR) method and domain-specific thresholds were applied:

- **Lower Bound:** Removed entries below the local minimum wage or statistically improbable entries (e.g., 0 or negative values).
- **Upper Bound:** Identified and capped "extreme" outliers—entries that fell outside the 1.5x IQR range—preventing high-earning anomalies from distorting the general trend.

1.4 Missing Value Imputation

A conservative approach to data loss was adopted. Rows with critical missing features (Salary or Experience) were dropped to preserve the accuracy of the training set, while non-critical missing values in categorical fields were filled with a "General/Other" placeholder to retain as much contextual data as possible.

Resulting Dataset Status

The final pre-processed dataset consists of cleaned, encoded, and normalized features. The reduction in noise and the standardization of the "Years of Experience" and "Job Role" variables provide a robust foundation for applying regression algorithms or deep learning models to predict salary trends within the specified market.

2. Introduction

2.1 Project Overview

The technology sector in Egypt is witnessing rapid growth, leading to a complex and ever-changing labor market. This project provides a data-driven analysis of the Egyptian tech ecosystem to bridge the information gap for employers and job seekers. By analyzing salary data, the project uncovers insights into job roles and the factors influencing earning potential.

2.2 Objectives

The primary objectives of this study are:

- **Market Insight Extraction:** Identifying in-demand job roles and the distribution of tech opportunities in Egypt.
- **Salary Benchmarking:** Establishing realistic salary ranges based on seniority, tech stacks, and specialized roles.
- **Predictive Modeling:** Developing a machine learning model to forecast salaries for candidates and HR professionals.

2.3 The Data Science Pipeline

The project follows a structured **Data Science Pipeline** to ensure accuracy and reliability:

1. **Data Acquisition:** Gathering and integrating data from multiple sources, resulting in 1,104 records.
2. **Exploratory Data Analysis (EDA):** Using statistical visualizations to identify patterns and correlations.
3. **Advanced Pre-processing:** Cleaning and engineering features to transform raw data into a machine-readable format.
4. **Modeling & Evaluation:** Training regression algorithms to determine the best architecture for salary prediction.

3. Dataset Description

The project utilizes a hybrid dataset constructed from two primary sources to capture both salary expectations and live market demands in Egypt.

3.1 Data Sources

-
- **Web Developers Salaries (Egypt 2024)**:

  A specialized dataset containing salary reports directly from developers, providing insights into actual paid compensations.

```python
# dataset 1
import pandas as pd

df1 = pd.read_csv('Web-Developers-Salaries-Egypt-2024.csv')
df1
```

✓ 0.0s Python

	Title	Salary	Years of Experiences	Work Hour	Work Type	What Is your Company	City of Company site
0	.Net Developer	15000	1	Full Time	Hybird	Egyptian	Alexandria
1	.Net Developer	311050	2	Full Time	Hybird	Egyptian	Cairo
2	.Net Developer	15000	1	Full Time	Hybird	Egyptian	Giza
3	.Net Developer	30000	2	Full Time	Hybird	Egyptian	Mansoura
4	.Net Developer	54000	3	Full Time	Hybird	Egyptian	un-known
...	...	...	...	...	...	...	...
191	Web Developer	5000	0	Full Time	On Site	Egyptian	Cairo
192	Web Developer	9000	1	Full Time	Remotley	Not Egyptian but site in egypt	Ksa
193	Wordpress Developer	14000	4	Full Time	Hybird	Egyptian	Cairo
194	Wordpress Developer	30000	3	Full Time	On Site	Egyptian	Cairo
195	Wordpress Developer	40000	3	Full Time	Remotley	Not Egyptian and site out of egypt	Uae

196 rows × 7 columns

```
df1.info()
```

[3] ✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 196 entries, 0 to 195
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Title                 196 non-null    object
 1   Salary                196 non-null    int64
 2   Years of Experiences  196 non-null    int64
 3   Work Hour             196 non-null    object
 4   Work Type             196 non-null    object
 5   What Is your Company  196 non-null    object
 6   City of Company site  196 non-null    object
dtypes: int64(2), object(5)
memory usage: 10.8+ KB
```

- **[Wuzzuf Job Listings (January 2025):]**

  Recent market data reflecting active job postings, required skills, and offered salary ranges on Egypt's leading recruitment platform.

```
df2 = pd.read_csv('wuzzuf_jobs.csv')
df2
```

Python

	Unnamed: 0	TimeStamp	Title	Years of Experiences	Salary	Date of Salary	What Is your Company	Work Type	Work Hour	City of Company site
0	0	2024-02-19 20:09:36.277000	Dotnet Developer	2	8000	2024-02-01 00:00:00	Egyption	Hybird	Full Time	Cairo
1	1	2024-02-19 20:10:43.450000	Android deve	NaN	6000	2024-02-06 00:00:00	Egyption	Hybird	Full Time	Assiut
2	2	2024-02-19 20:23:24.387000	Android Developer	1	10000	2024-02-20 00:00:00	Egyption	Hybird	Full Time	Cairo
3	3	2024-02-19 20:27:08.358000	DevOps Engineer	1 month	10000	2024-02-19 00:00:00	Egyption	On Site	Full Time	Assiut
4	4	2024-02-19 20:31:36.432000	Java	1	20000	2024-02-19 00:00:00	Egyption	On Site	Full Time	Cairo
...	...	...	...	...	...	...	...	...	...	...
903	1323	2024-03-07 10:04:23.532000	Software developer	4	45K	2024-02-01 00:00:00	Egyption	Remotley	Full Time	Cairo
904	1324	2024-03-07 12:25:44.049000	Embedded SW Engineer	0	10k	2024-03-01 00:00:00	Egyption	On Site	Full Time	cairo
905	1325	2024-03-07 12:52:53.200000	Php developer	1	10000	2024-03-01 00:00:00	Egyption	Remotley	Full Time	Cairo
906	1329	2024-03-08 19:01:07.101000	Seniors interior design	7	19000	2024-01-01 00:00:00	Egyption	Remotley	Full Time	Cairo
907	1330	10:10:49 2024/03/02 ص	DevOps Engineer	1	29000	2024-03-03 00:00:00	Egyption	Hybird	Full Time	cairo

908 rows × 10 columns

3.2 Feature Catalog

The integrated dataset includes the following key attributes:

- **Professional Identity:** `Job Title`, `Job Category`, and `Technical Stack` (e.g., Python, React, .NET).
- **Compensation Metrics:** `Salary` (Normalized to EGP), `Salary Period` (Monthly/Yearly), and `Currency`.
- **Experience & Seniority:** `Years of Experience` and `Career Level` (Junior, Mid, Senior, Lead).
- **Employment Metadata:** `Job Type` (Full-time/Part-time), `Work Mode` (Remote/On-site/Hybrid), and `Company Location` within Egypt.

3.3 Data Characteristics

- **Combined Volume**: After merging and initial filtering, the dataset provides a robust sample size for statistical significance.
- **Temporal Relevance:**

  The data spans from 2024 to early 2025, ensuring the results reflect current economic conditions and inflation adjustments in the Egyptian market.

4. Tech Stack & Methodology

This project leverages a robust suite of industry-standard tools and libraries tailored for large-scale data manipulation and machine learning.

4.1 Programming Language

- **Python:** The core language used for the entire pipeline, chosen for its extensive ecosystem in data science and its efficiency in handling complex data structures.

4.2 Data Manipulation & Visualization

- **Pandas:** Utilized as the primary tool for data ingestion, cleaning, and structural transformation (e.g., merging datasets and handling null values).
- **NumPy:** Used for high-performance mathematical operations and array management.
- **Matplotlib & Seaborn:** Employed for exploratory data analysis (EDA) to visualize salary distributions, correlations, and market trends.

4.3 Machine Learning Frameworks

- **Scikit-learn:** The foundational library for model building, providing essential tools for data splitting (Train-Test Split), feature scaling, and performance evaluation.
- **XGBoost:** Integrated to handle complex non-linear relationships in the data, offering high-performance gradient boosting for accurate salary forecasting.

4.4 Predictive Models

The project evaluates and compares multiple algorithms to identify the most accurate predictor for the Egyptian market:

- **Random Forest Regressor:** An ensemble learning method used to capture intricate patterns by building multiple decision trees.

- **XGBoost Regressor:** Optimized for speed and performance, providing superior accuracy in regression tasks.
- **Logistic Regression:** (Used for classification tasks within the data, such as predicting job categories or salary brackets).

5. Data Preprocessing & Integration Pipeline

This stage focuses on refining the raw data into a high-quality format. The pipeline ensures that the model learns from realistic patterns rather than noise.

5.1 Data Integration & Localization

- **Source Merging:** Combined the "Web Developers Salaries" and "Wuzzuf Job Listings" datasets into a unified structure.

```
# Clean column names by removing extra spaces
df1.columns = df1.columns.str.strip()
df2.columns = df2.columns.str.strip()

# Concatenate the dataframes after cleaning
df = pd.concat([df1, df2], ignore_index=True)
```

[8] ✓ 0.0s

```
# information about dataset after integration
rows, cols = df.shape
print("Rows:", rows)
print("Columns:", cols)
```

[9] ✓ 0.0s

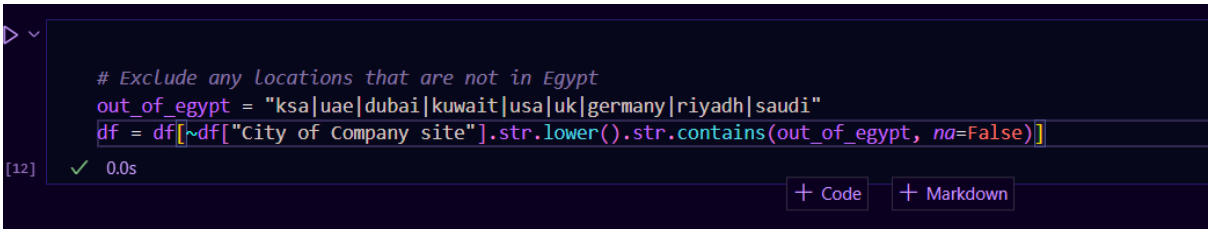
```
Rows: 1104
Columns: 10
```

```
df.info()
```

[10] ✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1104 entries, 0 to 1103
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Title                 1104 non-null   object
 1   Salary                1104 non-null   object
 2   Years of Experiences  1103 non-null   object
 3   Work Hour             1104 non-null   object
 4   Work Type             1104 non-null   object
 5   What Is your Company  1104 non-null   object
 6   City of Company site  1012 non-null   object
 7   Unnamed: 0            908 non-null    float64
 8   TimeStamp             908 non-null    object
 9   Date of Salary        908 non-null    object
dtypes: float64(1), object(9)
memory usage: 86.4+ KB
```

Geographic Filtering: Strictly retained records within Egypt, filtering out international roles to ensure the model reflects the local economy.




```
# Exclude any locations that are not in Egypt
out_of_egypt = "ksa|uae|dubai|kuwait|usa|uk|germany|riyadh|saudi"
df = df[~df["City of Company site"].str.lower().str.contains(out_of_egypt, na=False)]
```

Automated Cleaning & Standardization

To handle the inconsistencies found in raw crowdsourced data, a specialized cleaning pipeline was developed in Python:

- **Dynamic Experience Parsing:** A custom function was implemented to normalize professional tenure. This includes converting monthly values to yearly decimals and calculating the mean for range-based entries (e.g., transforming "1-3 years" to 2.0).
- **Target Variable Integrity:** The `salary` feature was strictly cast to numeric types, and all records with null target values were pruned to ensure the model learns only from verified data points.
- **Statistical Imputation:** Missing values in the `Years of Experience` column were addressed using the **Median** imputation strategy, preserving the central tendency of the dataset without introducing outlier-driven bias.
- **Structural Normalization:** Applied uniform string manipulation (lowercase and strip) across all column headers and categorical features to eliminate duplication caused by formatting variance

```
--- Data Types ---
title                    object
salary                  float64
years of experiences    float64
work hour                object
work type                object
what is your company     object
city of company site     object
unnamed: 0              float64
timestamp                object
date of salary           object
dtype: object

--- Missing Values ---
title                     0
salary                    0
years of experiences      0
work hour                 0
work type                 0
what is your company      0
city of company site     74
unnamed: 0              165
timestamp               165
date of salary          165
dtype: int64

Final Shape: (914, 10)
```

5.2 Handling Missing Values

After merging the datasets, we identified several columns with missing data. The primary goal was to achieve **zero missing values** in the final training set to ensure model stability.

5.2.1 Affected Columns

The following features were identified with missing entries:

- **city of company site:** Approximately **7.5%** missing values.
- **years of experiences:** Contained inconsistent text and null values.
- **Metadata (timestamp, unnamed: 0, date of salary):** Contained approximately **18.8%** missing values.

```python
import pandas as pd
import numpy as np

# 1. Handling Categorical Missing Values (City/Location)
# Since 'city of company site' is categorical, we fill missing values with 'Unknown'
# or the 'Mode' (most frequent city) to avoid losing 7.5% of the data.
df['city of company site'] = df['city of company site'].fillna(df['city of company site'].mode()[0])

# 2. Handling Redundant/Metadata Columns
# Columns like 'unnamed: 0', 'timestamp', and 'date of salary' have ~18.8% missing values.
# These columns do not provide predictive value for a salary model, so we drop them.
cols_to_drop = ['unnamed: 0', 'timestamp', 'date of salary']
df = df.drop(columns=[col for col in cols_to_drop if col in df.columns])

# 3. Final Verification
# Ensure all critical columns have 0 missing values
print("--- Missing Values After Targeted Fix ---")
print(df.isnull().sum())
print(f"\nFinal Dataset Shape: {df.shape}")
```

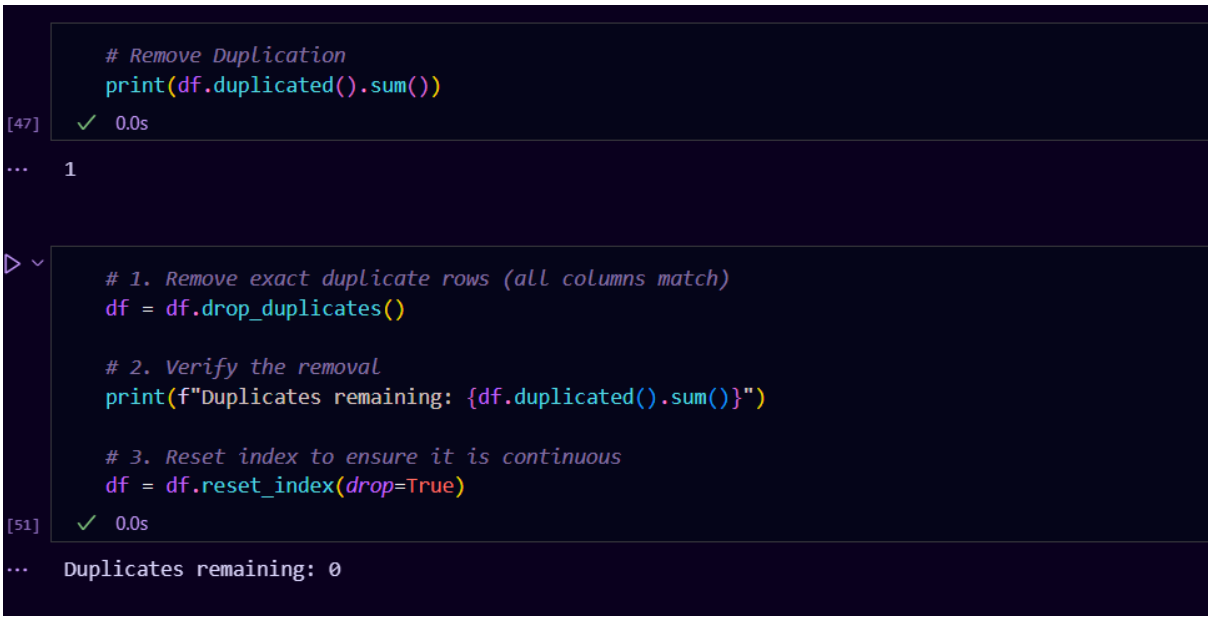
✓ 0.0s

```
--- Missing Values After Targeted Fix ---
title                   0
salary                  0
years of experiences    0
work hour               0
work type               0
what is your company    0
city of company site    0
dtype: int64

Final Dataset Shape: (852, 7)
```

Data Deduplication

During the data integrity check, we identified duplicate records that could potentially bias the model's learning process.

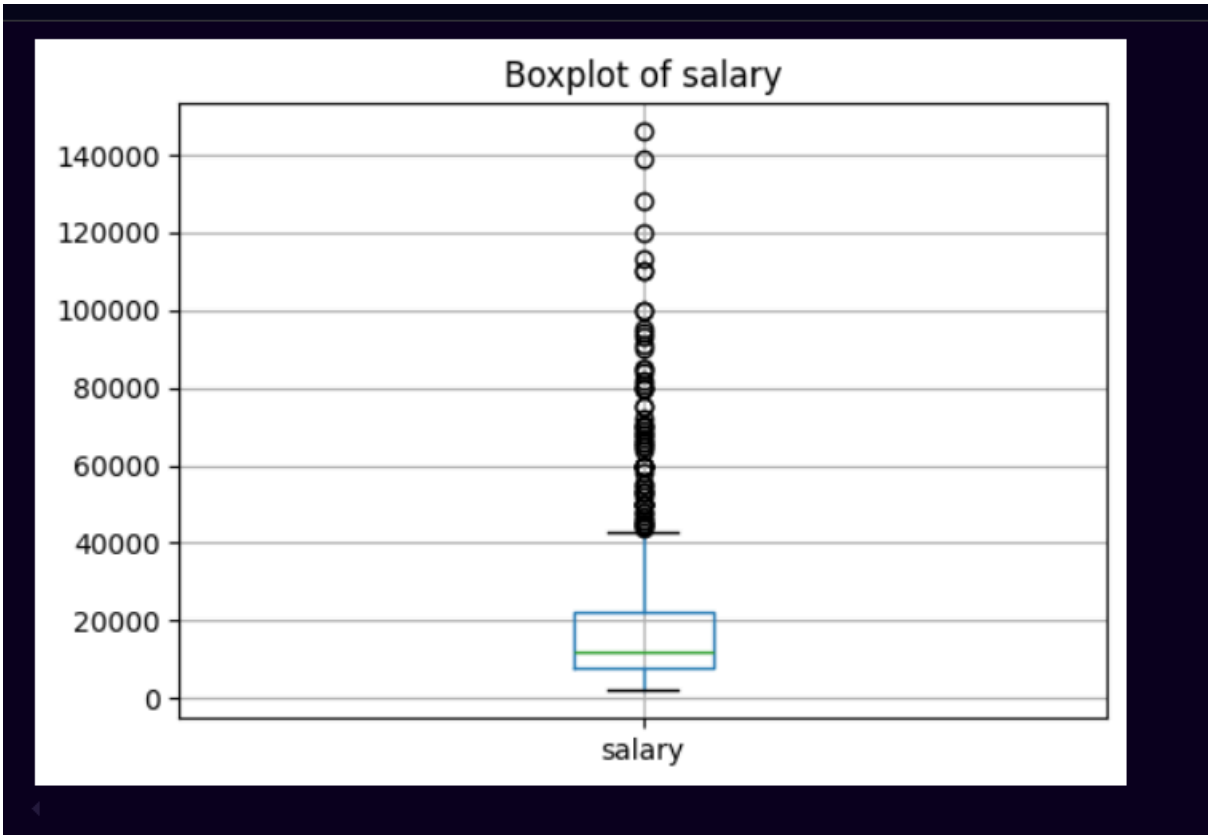
- **Detection:** By using the `df.duplicated()` method, we found **1 exact duplicate row** where all feature values were identical across the entire dataset.
- **Action:** To resolve this, we utilized the `drop_duplicates()` function to remove the redundant entry and retain only the unique records.



Outlier Analysis Results

After applying the Grouped IQR method, we identified **146 outlier records** across the dataset.

- **Column Distribution:** Outliers were primarily detected in salary and years of experiences, representing extreme values that sit outside the $1.5 \times IQR$ range for their respective job titles.
- **Example Case:** In the .net developer group, salaries exceeding 120k EGP were flagged, while in the software engineer group, salaries as low as 2.8k EGP were captured.
- **Action Taken:** These records were reviewed and removed to ensure the regression model is trained on the most representative data points of the Egyptian tech market.



TOTAL OUTLIERS FOUND: 146

	title	salary	years of experiences	work type	what is your company	city of company site	salary_year	salary_month	outlier_column	title_group
0	.net developer	128000.0	4.0	Hybird	Not Egyptian but site in egypt	Cairo	2024	2	salary	.net developer
1	.net developer	128000.0	4.0	Hybird	Not Egyptian but site in egypt	Cairo	2024	2	years of experiences	.net developer
2	.net developer	37500.0	7.0	On Site	Egyptian	Alexandria	2024	2	years of experiences	.net developer
3	.net developer	45000.0	4.0	Hybird	Egyption	Cairo	2024	2	years of experiences	.net developer
4	.net developer	30000.0	4.0	On Site	Egyption	Cairo	2024	2	years of experiences	.net developer
...	...	...	...	...	...	...	...	...	...	...
141	software engineer	29000.0	3.0	On Site	Egyption	Cairo	2024	1	salary	software engineer
142	software engineer	12000.0	0.0	Hybird	Egyption	Cairo	2024	1	salary	software engineer
143	software engineer	2800.0	0.0	Remotley	Egyption	Cairo	2022	9	salary_year	software engineer
144	software engineer	15000.0	2.0	Hybird	Egyption	Cairo	2023	5	salary_year	software engineer
145	software engineer	2800.0	0.0	Remotley	Egyption	Cairo	2022	9	salary_month	software engineer

146 rows × 10 columns

Handling Outliers using IQR Clipping

In this step, outliers were treated using the IQR (Interquartile Range) method with clipping instead of removal.

- For each numerical column:
  - Calculated Q1 (25th percentile) and Q3 (75th percentile)
  - Computed IQR = Q3 - Q1
  - Defined lower and upper bounds:
    - Lower = Q1 - 1.5 * IQR
    - Upper = Q3 + 1.5 * IQR

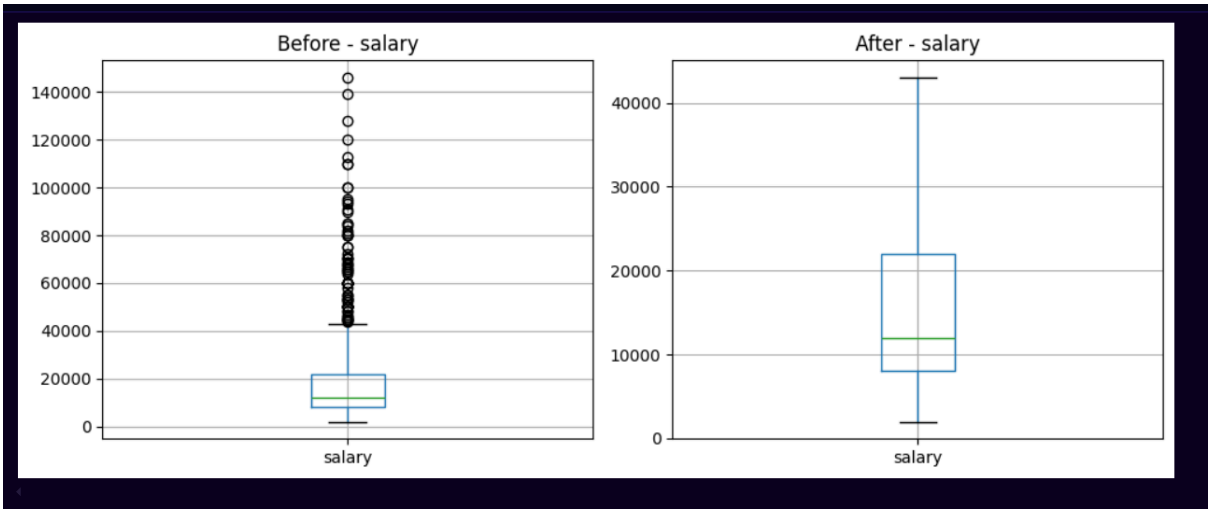
- Values outside this range were clipped to the nearest boundary.
 This approach preserves all data points while reducing the impact of extreme values.

Dataset shape remains unchanged before and after clipping.

```
import pandas as pd


df_fixed = df.copy()



num_cols = df_fixed.select_dtypes(include=['number']).columns


for col in num_cols:
    Q1 = df_fixed[col].quantile(0.25)
    Q3 = df_fixed[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df_fixed[col] = df_fixed[col].clip(lower, upper)


print("Before shape:", df.shape)
print("After fixing outliers:", df_fixed.shape)
```

3] ✓ 0.0s

```
Before shape: (851, 7)
After fixing outliers: (851, 7)
```


Before - salary
140000
120000
100000
80000
60000
40000
20000
0
salary
After - salary
40000
30000
20000
10000
0
salary

6. Statistical Methodology & Data Audit

6. Statistical Methodology & Data Audit

In this phase, we performed a rigorous statistical audit to measure the impact of our preprocessing pipeline. Understanding these metrics is crucial for validating that the data is ready for predictive modeling.

6.1 Statistical Metrics: Definitions and Importance

1. Mean (The Arithmetic Average)

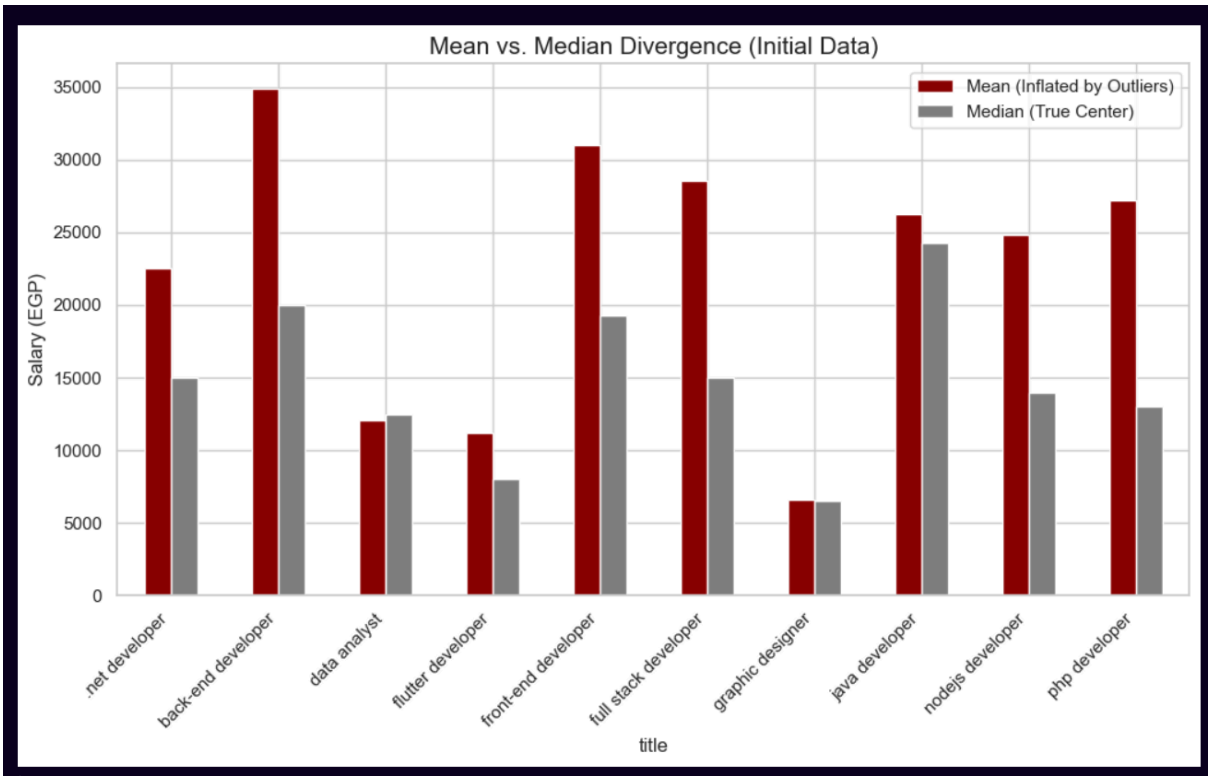
- **Definition:** The sum of all values divided by the total number of observations $\mu = \frac{\sum X_i}{N}$.
- **Importance:** It provides a general baseline for salaries within a specific job title.
- **Project Context:** We monitored the shift in the Mean before and after cleaning; a significant drop (as seen in .net developer) indicated the successful removal of inflated outliers.

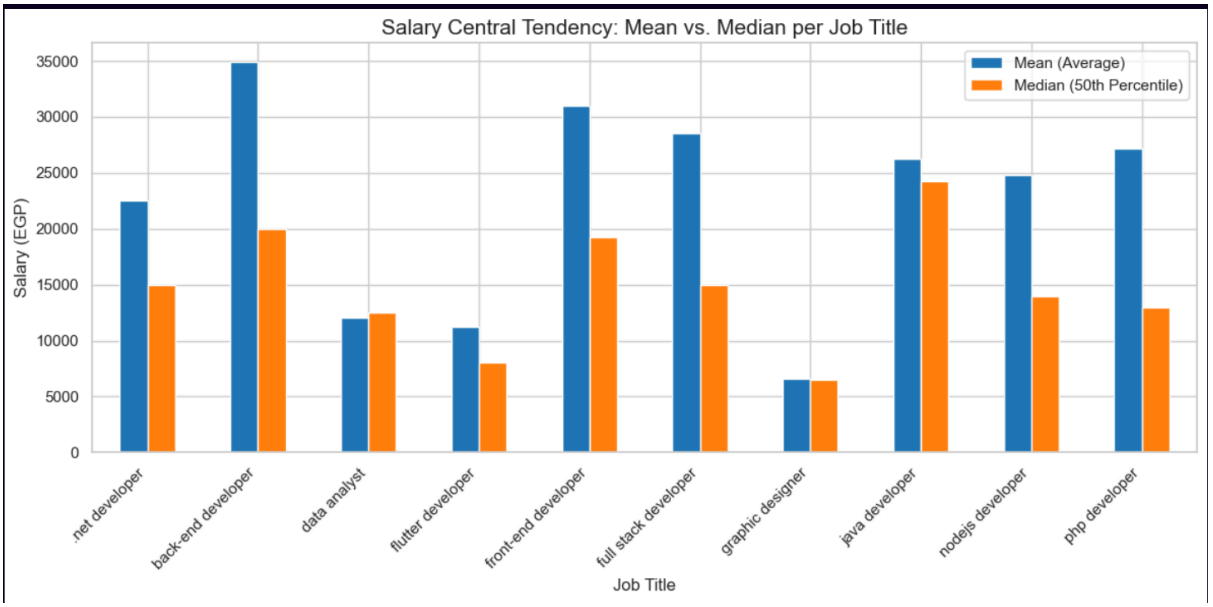
2. Median (The 50th Percentile)

- **Definition:** The middle value in a dataset when ordered from lowest to highest.
- **Importance:** Unlike the Mean, the Median is **Robust to Outliers**. It represents what the "typical" employee earns.
- **Project Context:** We used the gap between Mean and Median as a "Skewness Indicator." The closer they are, the more balanced the data.

3. Standard Deviation (std)

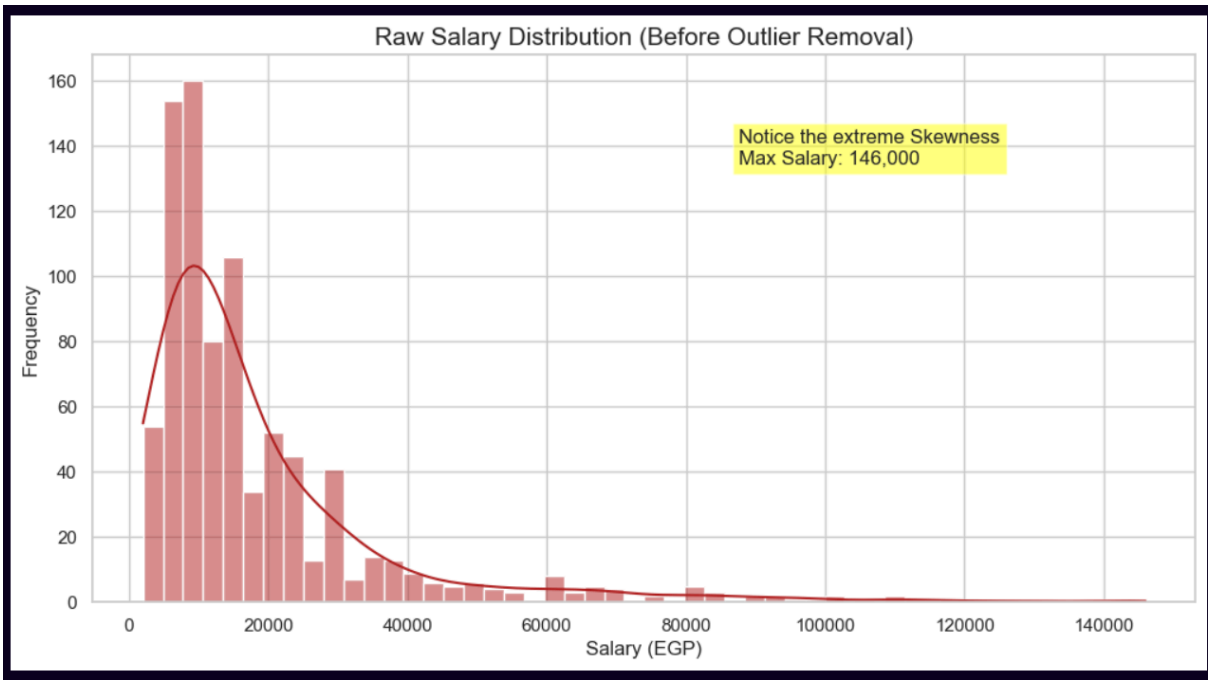
- **Definition:** Measures the amount of variation or dispersion of a set of values $\sigma = \sqrt{\frac{\sum(X_i - \mu)^2}{N}}$.
- **Importance:** It quantifies "Noise." A high std means salaries are volatile and unpredictable.
- **Project Context:** Our goal was to **minimize std**. For instance, reducing the std of .net developers from **21,228** to **12,906** significantly improved the data's reliability for the regression model.

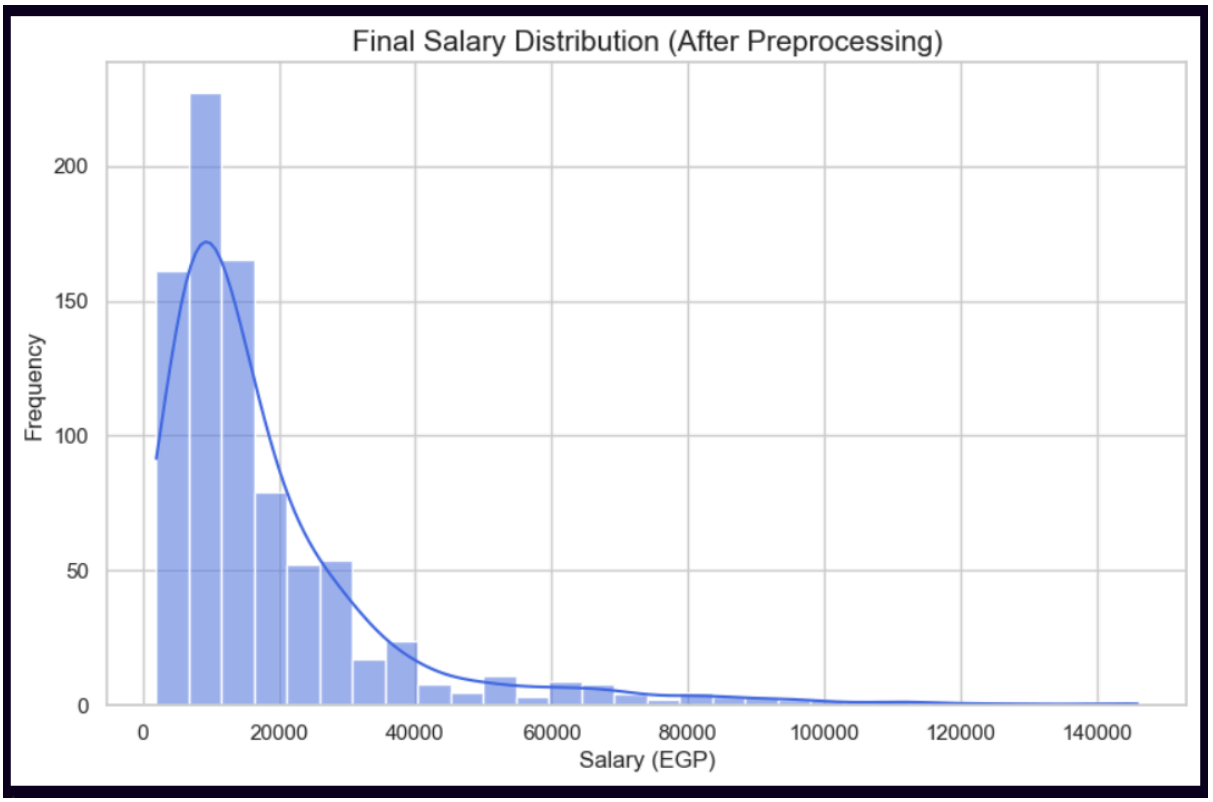

Mean vs. Median Divergence (Initial Data)
Mean (Inflated by Outliers)
Median (True Center)
Salary (EGP)
35000
30000
25000
20000
15000
10000
5000
0
.net developer
back-end developer
data analyst
flutter developer
front-end developer
full stack developer
graphic designer
java developer
nodejs developer
php developer
title


Salary Central Tendency: Mean vs. Median per Job Title
Mean (Average)
Median (50th Percentile)
Salary (EGP)
35000
30000
25000
20000
15000
10000
5000
0
.net developer
back-end developer
data analyst
flutter developer
front-end developer
full stack developer
graphic designer
java developer
nodejs developer
php developer
Job Title

4. Skewness

- **Definition:** A measure of the asymmetry of the probability distribution.
- **Importance:** Most Machine Learning models (like Linear Regression) perform best when data is **Normally Distributed** (Skewness ≈ 0).
- **Project Context:** High positive skewness (e.g., **3.10** in raw data) meant the data was "tailing" towards extreme high salaries. Our preprocessing successfully brought this down to **0.67**, making the distribution more symmetric.





5. Variance (var)

- **Definition:** The average of the squared differences from the Mean.
- **Importance:** It represents the total spread of the data. While std is easier to read, Variance is used internally by models to calculate error.

Before Preprocessing

title	mean	median	skew	max	min	count	std	var
.net developer	22510.227273	15000.0	3.101586	128000.0	5500.0	44	21228.331167	4.506420e+08
front-end developer	30971.052632	19250.0	0.990412	93000.0	3500.0	38	28257.779679	7.985021e+08
full stack developer	28575.000000	15000.0	2.441538	146000.0	5000.0	20	35457.637808	1.257244e+09
php developer	27175.000000	13000.0	1.668813	100000.0	5000.0	20	29810.486940	8.886651e+08
java developer	26225.000000	24250.0	1.312409	81000.0	4000.0	20	19385.103478	3.757822e+08
flutter developer	11194.117647	8000.0	1.302798	27500.0	4000.0	17	7002.898560	4.904059e+07
back-end developer	34866.666667	20000.0	2.235088	139000.0	7000.0	15	35904.370340	1.289124e+09
data analyst	12083.333333	12500.0	0.501880	24000.0	4500.0	12	5810.778908	3.376515e+07
graphic designer	6590.909091	6500.0	0.164274	10000.0	3500.0	11	2142.640682	4.590909e+06
full stack .net developer	23727.272727	15000.0	3.068236	120000.0	5000.0	11	32622.357086	1.064218e+09
nodejs developer	24818.181818	14000.0	1.866681	85000.0	7000.0	11	29208.965000	8.531636e+08
android developer	20700.000000	18000.0	1.683720	50000.0	7000.0	10	12156.845351	1.477889e+08

After Preprocessing

title	mean	median	skew	max	min	count	std	var
.net developer	20020.454545	15000.0	0.673519	43000.0	5500.0	44	12906.003282	1.665649e+08
front-end developer	22576.315789	19250.0	0.318954	43000.0	3500.0	38	14707.222609	2.163024e+08
full stack developer	19825.000000	15000.0	0.756035	43000.0	5000.0	20	13948.471903	1.945599e+08
java developer	23375.000000	24250.0	0.140203	43000.0	4000.0	20	13432.212852	1.804243e+08
php developer	20075.000000	13000.0	0.658783	43000.0	5000.0	20	15242.146382	2.323230e+08
flutter developer	11194.117647	8000.0	1.302798	27500.0	4000.0	17	7002.898560	4.904059e+07
back-end developer	25066.666667	20000.0	0.182367	43000.0	7000.0	15	13090.163519	1.713524e+08
data analyst	12083.333333	12500.0	0.501880	24000.0	4500.0	12	5810.778908	3.376515e+07
graphic designer	6590.909091	6500.0	0.164274	10000.0	3500.0	11	2142.640682	4.590909e+06
full stack .net developer	16727.272727	15000.0	1.429912	43000.0	5000.0	11	10982.630915	1.206182e+08
nodejs developer	17454.545455	14000.0	1.632609	43000.0	7000.0	11	13056.520489	1.704727e+08
android developer	20000.000000	18000.0	1.220824	43000.0	7000.0	10	10349.449798	1.071111e+08
data engineer	20222.222222	21000.0	-0.200049	30000.0	11000.0	9	6379.219736	4.069444e+07
devops engineer	21862.500000	21000.0	0.060532	34000.0	10000.0	8	8277.842628	6.852268e+07
accountant	8950.000000	7750.0	1.682401	18000.0	5000.0	8	4219.681775	1.780571e+07
pharmacist	4762.500000	4700.0	0.828539	8700.0	2000.0	8	2021.270817	4.085536e+06
data scientist	21714.285714	20000.0	0.302763	40000.0	7000.0	7	11441.361890	1.309048e+08
software engineer	14971.428571	15000.0	0.478459	29000.0	2800.0	7	7690.191650	5.913905e+07
machine learning engineer	21666.666667	19000.0	0.413484	35000.0	10000.0	6	11236.844160	1.262667e+08
frontend developer vue.js	21000.000000	14000.0	0.541697	43000.0	5000.0	5	17392.527131	3.025000e+08
medical representative	7700.000000	8000.0	-0.382400	10000.0	4500.0	5	2439.262184	5.950000e+06
full-stack developer	14600.000000	14000.0	0.280036	25000.0	5000.0	5	7092.249291	5.030000e+07
javascript developer	24800.000000	24000.0	0.196043	30000.0	20000.0	5	5019.960159	2.520000e+07
network engineer	18000.000000	10000.0	2.007235	43000.0	10000.0	5	14300.349646	2.045000e+08
react developer	10400.000000	10000.0	1.492685	18000.0	6000.0	5	4560.701700	2.080000e+07
sales	13000.000000	5000.0	2.102952	43000.0	3000.0	5	16985.287751	2.885000e+08
software developer	16200.000000	12000.0	0.462552	27000.0	7000.0	5	9148.770409	8.370000e+07

7.**Visualization**

7.1. Analysis of Top 10 Job Roles

Objective:

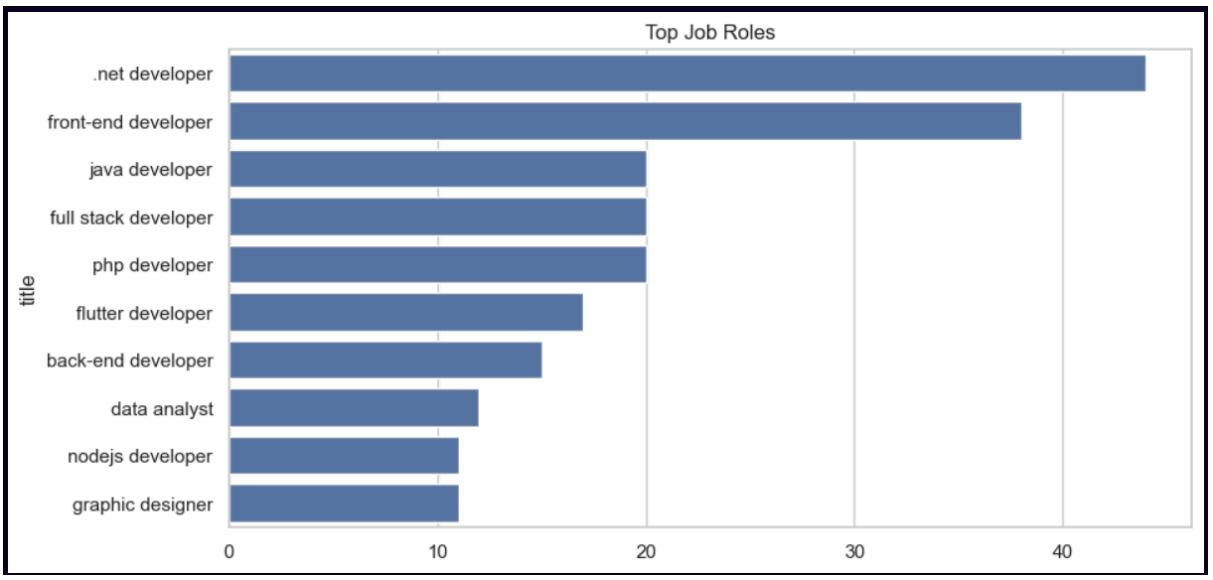
The primary goal of this visualization is to identify the most frequent job titles within the dataset. By isolating the top 10 roles, we can understand which positions are most prevalent in the current labor market sample and identify the dominant professional domains.

Technical Implementation:

- **Data Aggregation:** The `value_counts()` function is utilized to calculate the frequency of each unique title, followed by `.head(10)` to filter for the most occurring roles.
- **Visualization Choice:** A **Horizontal Bar Plot** (`sns.barplot`) was selected to ensure that long job titles remain legible on the Y-axis.
- **Aesthetics:** The figure size was set to `(10, 5)` to provide a clear, uncluttered view of the distribution.

Key Insights & Utility:

- **Market Demand:** Highlights which tech stacks or roles (e.g., .NET, Frontend, Data Science) have the highest representation.
- **Data Density:** Helps determine if the dataset is skewed towards a specific niche or if it offers a balanced variety of roles.
- **Prioritization:** This analysis guides further exploration, such as identifying which roles deserve a deeper dive into their specific salary benchmarks.



7.2. Job Roles Proportion (Top 5 Distribution)

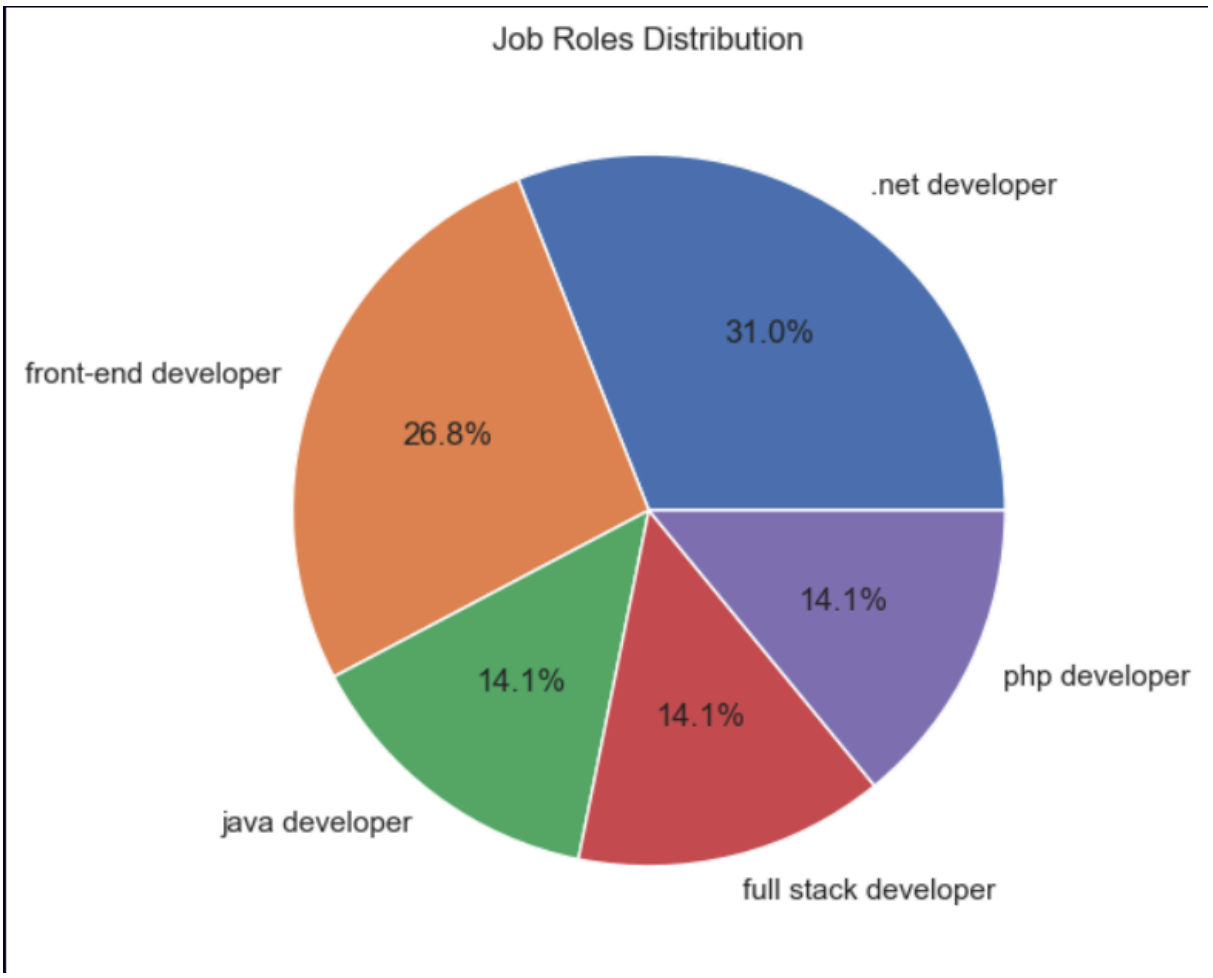
Objective: The purpose of this visualization is to illustrate the relative market share of the top 5 most common job roles. While the bar chart shows absolute counts, the pie chart focuses on the **percentage contribution** of each key role to the overall top-tier segment of the dataset.

Technical Implementation:

- **Data Selection:** The code extracts the top 5 job titles using `value_counts().head(5)`. Limiting the data to five categories ensures the chart remains clean and prevents overcrowding with small, illegible slices.
- **Visualization Choice:** A **Pie Chart** (`plt.pie`) was implemented to emphasize "part-to-whole" relationships.

Key Insights & Utility:

- **Market Dominance:** Clearly identifies if a single role dominates a large portion of the dataset (e.g., if one role occupies 40% or more of the top 5).
- **Comparison:** Provides a quick visual comparison of how the top roles scale against each other in terms of volume.



7.3. Salary Distribution Analysis

Objective: The aim of this visualization is to explore the distribution and spread of salaries within the dataset. It helps in understanding the concentration of wealth among employees and identifying the presence of outliers or skewness in the financial data.

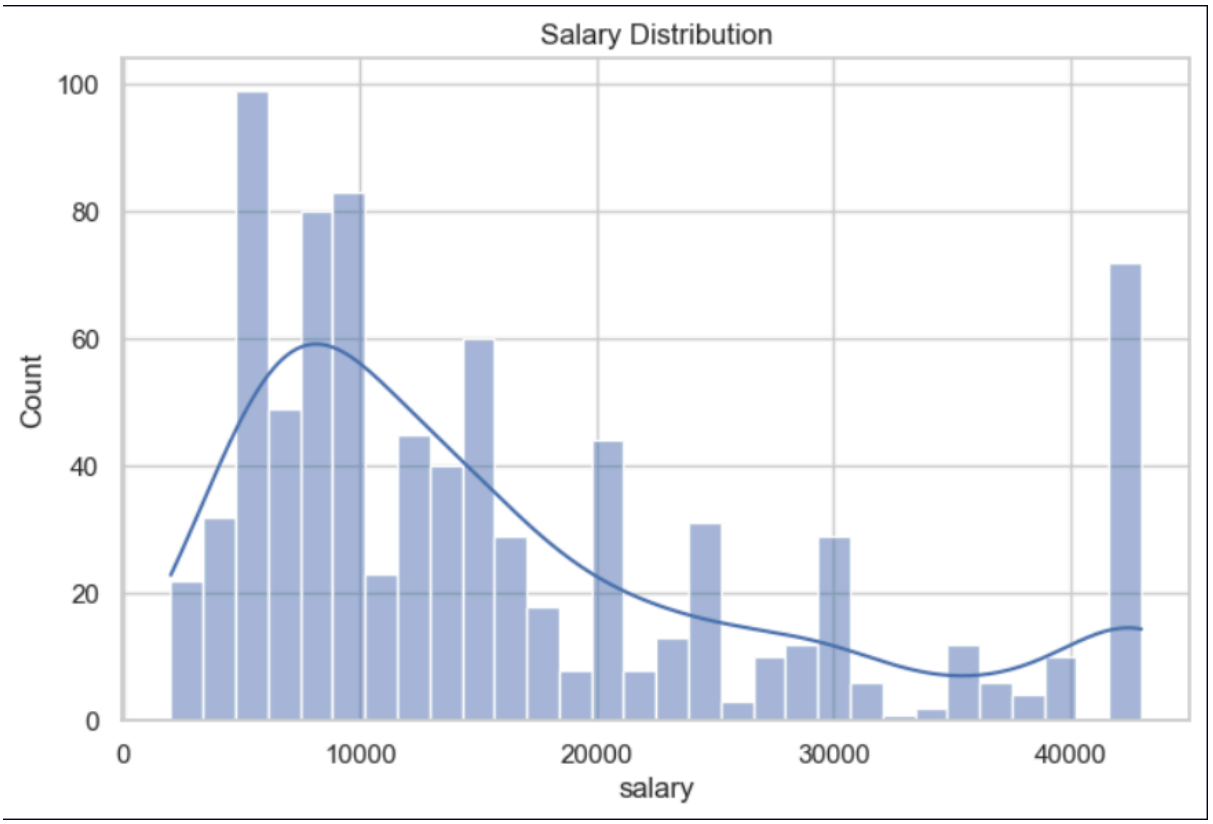
Technical Implementation:

- **Visualization Choice:** A **Histogram** combined with a **Kernel Density Estimate (KDE)** was used.
    - The histogram segments the salary range into **30 bins**, providing a granular view of salary frequency.
    - The **KDE plot** (the smooth line) overlays the histogram to visualize the probability density, making it easier to identify the shape of the distribution.

- **Library:** Built using `seaborn` (`sns.histplot`) for its high-level interface and statistical accuracy.

Key Insights & Utility:

- **Central Tendency:** Quickly identifies the most common salary ranges (mode).
- **Skewness:** Helps determine if the distribution is "Right-Skewed" (many low salaries, few very high) or "Left-Skewed."
- **Outlier Detection:** Highlights extreme salary values that might require further investigation or specialized handling during the modeling phase.



7.4. Salary Benchmarking by Job Title (Top 20 Roles)

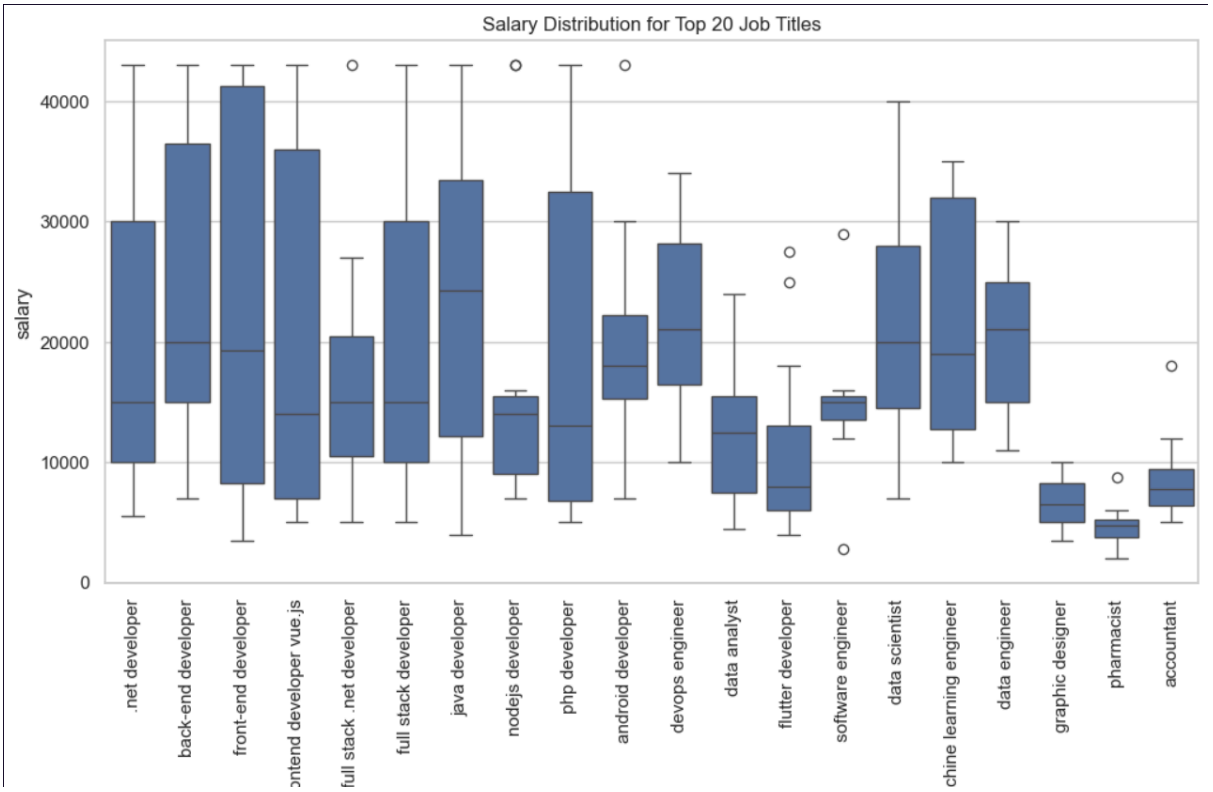
Objective: The goal of this visualization is to perform a comparative analysis of salary ranges across the most frequent 20 job titles. It allows us to identify which roles command higher pay scales and observe the pay variance within each specific profession.

Technical Implementation:

- **Data Filtering:** To maintain clarity, the dataset was filtered to include only the **top 20 job titles** based on frequency.
- **Visualization Choice:** A **Box Plot** (`sns.boxplot`) was selected because it effectively summarizes the distribution of a numerical variable (salary) across multiple categories (job titles).
- **Statistical Elements:**
    - **Medians:** The line within each box represents the median salary for that role.
    - **Interquartile Range (IQR):** The box itself shows where the middle 50% of salaries lie.
    - **Outliers:** Individual points beyond the "whiskers" highlight unusual salary entries that deviate significantly from the norm for that specific role.
- **Aesthetics:** `plt.xticks(rotation=90)` was applied to rotate the job titles vertically, ensuring all labels are legible.

Key Insights & Utility:

- **Market Comparison:** Directly compares the earning potential of different roles (e.g., comparing a ".NET Developer" vs. a "Project Manager").
- **Pay Stability:** A "short" box indicates consistent pay across the role, while a "long" box suggests high salary variance based on company or individual skill.
- **Data Quality Check:** Helps identify potential data entry errors or extreme cases (outliers) that might need cleaning.



7.5. Correlation Analysis: Salary vs. Experience per Job Role

Objective:

The goal of this analysis is to quantify the strength of the relationship between **Years of Experience** and **Salary** for the top 10 most frequent job titles. This helps determine if professional experience is a consistent predictor of income within specific technical domains.

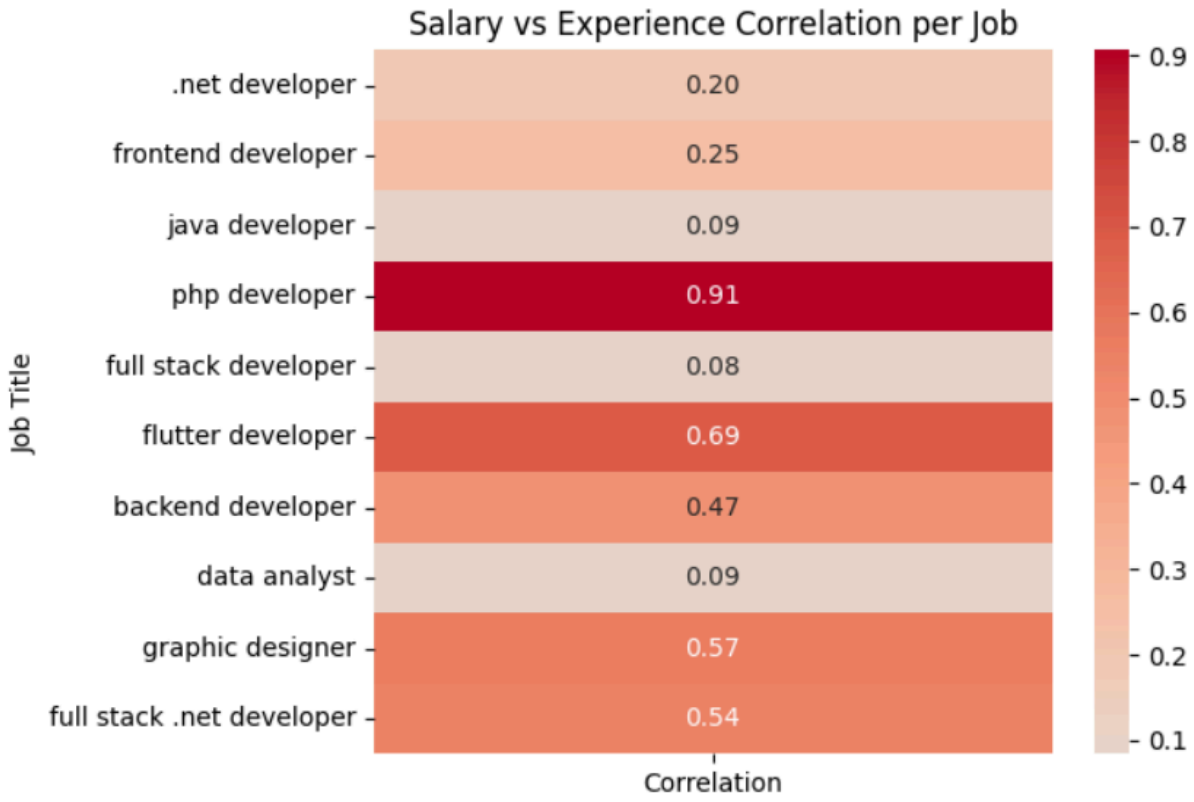
Technical Implementation:

- **Data Preprocessing:** Columns are explicitly converted to numeric types using pd.to_numeric with errors="coerce" to handle inconsistencies. Rows with null values in critical features (Salary, Experience, Title) are dropped to ensure mathematical accuracy.
- **Segmented Correlation:** Instead of calculating a global correlation, the code iterates through each job title to calculate a unique correlation coefficient (r). This accounts for the fact that experience might be rewarded differently in different roles (e.g., a Manager vs. a Support Specialist).
- **Visualization Choice:** A **Heatmap** (sns.heatmap) is utilized to represent the correlation values.

- annot=True: Displays the correlation coefficient (ranging from -1 to 1) within each cell.
- cmap="coolwarm": Uses a diverging color scale to visually distinguish between strong positive correlations (warm/red) and weak or negative correlations (cool/blue).

Key Insights & Utility:

- **Predictive Validity:** Roles with high correlation (close to 1.0) indicate a structured career path where experience directly translates to higher pay.
- **Market Anomalies:** Low correlation values might suggest that other factors—such as specific technical skills, certifications, or company size—influence salary more than mere years of experience.
- **Model Selection:** This insight helps in selecting features for future Machine Learning models (e.g., knowing if "Experience" should be a primary feature for predicting a specific job's salary).



8. Statistical Hypothesis Testing

8. Statistical Hypothesis Testing (ANOVA) Results

Test Execution & Data Validation:

- **Excluded Groups:** The "backend developer" role was excluded from the test due to insufficient data points, ensuring the integrity of the statistical comparison.
- **Sample Distribution:** The test was conducted across three validated groups: Data Scientist (7 samples), Data Engineer (9 samples), and .NET Developer (44 samples).

Findings:

- **F-statistic:** 0.0597
- **P-value:** 0.9421
- **Statistical Significance:** The result is **not statistically significant** as the P-value is much higher than the standard threshold of 0.05.

Interpretation & Insights:

- **Accepting the Null Hypothesis:** We fail to reject the null hypothesis, meaning that within this specific dataset, the **Job Title** does not appear to be a primary driver of salary differences for these roles.
- **Potential Factors:** This outcome suggests that other variables—such as **Years of Experience**, **Company Size**, or **Location**—might have a more dominant influence on salary than the job title alone.
- **Data Limitation Note:** It is important to note that the small sample sizes for "Data Scientist" and "Data Engineer" compared to ".NET Developer" may affect the sensitivity of the test. A larger and more balanced dataset might yield different results.

```
Warning: backend developer excluded due to insufficient data.
Validated group for data scientist: 7 samples.
Validated group for data engineer: 9 samples.
Validated group for .net developer: 44 samples.

--- ANOVA Test Results ---
F-statistic: 0.0597
P-value: 0.9421
Result: Not Statistically Significant (No clear evidence that titles influence salary).
```

8.1. Segmented ANOVA: Role Comparison per Experience Level

Objective: To conduct a more granular analysis, this step performs the ANOVA test within each **Experience Level** (Junior, Mid, Senior) separately. This eliminates the "experience bias," allowing us to determine if job titles impact salary when candidates are at the same stage in their careers.

Technical Implementation:

- **Stratification:** The dataset is partitioned into three subsets based on the `experience level` feature created during Feature Engineering.
- **Targeted Comparison:** Within each level, we isolate the salary data for four key roles: Backend Developer, Data Scientist, Data Engineer, and .NET Developer.
- **Robustness Check:** The code dynamically checks for `len(groups) >= 2` to ensure that an ANOVA is only attempted when at least two different job roles have sufficient data within that specific experience bracket.

Key Utility:

- **Isolating Variables:** By holding "Experience" constant, we can see the pure effect of "Job Title" on salary.
- **Granular Insights:** It helps identify if certain roles (e.g., Data Science) command a "premium" at the entry-level but perhaps equalize with other roles at the senior level.

- **Data Sufficiency Check:** This approach also highlights where our dataset lacks representation (e.g., if we have many Junior .NET devs but few Senior ones).

```
=== JUNIOR LEVEL ===
F-statistic: 0.5852946291065939
P-value: 0.5629781109389091
 No significant difference between roles

=== MID LEVEL ===
F-statistic: 0.12738733492400775
P-value: 0.8810677199030003
 No significant difference between roles

=== SENIOR LEVEL ===
Not enough data for ANOVA
```

9. Effect Size Analysis (Cohen's d)

Objective:

While previous statistical tests (ANOVA) focused on the *existence* of a difference, this analysis uses **Cohen's d** to measure the **magnitude** of the difference between the salaries of "Data Scientists" and "Backend Developers." This helps quantify the practical significance of the salary gap.

Technical Implementation:

- **Metric: Cohen's d**, which is calculated as the difference between two means divided by their pooled standard deviation.
- **Formula Applied:**

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{pooled}}$$

- **Comparison Groups:** The function compares the salary distribution of the data scientist group against the backend developer group.

Interpretation Scale:

- **Small Effect (d ≈ 0.2):** Minimal practical difference.
- **Medium Effect (d ≈ 0.5):** Moderate difference that is noticeable.
- **Large Effect (d ≥ 0.8):** Significant practical difference in earning potential between the two roles.

Key Utility:

This provides a more nuanced view than the P-value. Even if a difference is statistically significant, Cohen's d tells us if that difference is large enough to matter in a real-world career decision.

10. Statistical Inference: 95% Confidence Intervals for Salary Means

Objective:

The purpose of this analysis is to estimate the precision of the average salary for each job title. Since our dataset is a sample of the overall market, **Confidence Intervals (CI)** provide a range within which we are 95% certain the true population mean lies.

Technical Implementation:

- **Statistical Distribution:** The **t-distribution** (st.t.interval) was used, which is the standard approach when dealing with sample-based estimates and unknown population variances.
- **Thresholding:** A minimum sample size of **n >= 5** was enforced per job title to ensure the calculated intervals are statistically meaningful and to reduce volatility in the results.
- **Parameters:**
    - Confidence Level: 0.95 (95%).
    - Standard Error of the Mean (SEM): Used to scale the interval based on the variability of salaries within each role.

Key Insights & Interpretation:

- **Reliability:** Narrower intervals (where ci_lower and ci_upper are close) indicate highly reliable mean estimates with low salary variance.
- **Overlap Analysis:** If the intervals of two different roles overlap significantly, it further reinforces our ANOVA findings that the difference in their salaries is likely not statistically significant.
- **Market Prediction:** These intervals provide a more realistic "salary bracket" for HR planning or job seekers than a single average number.

	title	mean	ci_lower	ci_upper
1	back-end developer	25066.666667	17817.575734	32315.757600
8	javascript developer	24800.000000	18566.896198	31033.103802
7	java developer	23375.000000	17088.530875	29661.469125
2	front-end developer	22576.315789	17742.172011	27410.459568
12	devops engineer	21862.500000	14942.050377	28782.949623
17	data scientist	21714.285714	11132.790815	32295.780613
19	machine learning engineer	21666.666667	9874.321926	33459.011408
3	frontend developer vue.js	21000.000000	-595.674776	42595.674776
20	data engineer	20222.222222	15318.719858	25125.724586
10	php developer	20075.000000	12941.455909	27208.544091
0	.net developer	20020.454545	16096.670912	23944.238179
11	android developer	20000.000000	12596.449614	27403.550386
5	full stack developer	19825.000000	13296.914202	26353.085798
22	network engineer	18000.000000	243.770683	35756.229317
9	nodejs developer	17454.545455	8683.055638	26226.035271
4	full stack .net developer	16727.272727	9349.040884	24105.504570
18	software developer	16200.000000	4840.301155	27559.698845
16	software engineer	14971.428571	7859.187520	22083.669623
6	full-stack developer	14600.000000	5793.809390	23406.190610
25	sales	13000.000000	-8090.020300	34090.020300
13	data analyst	12083.333333	8391.340556	15775.326111
14	flutter developer	11194.117647	7593.559448	14794.675846
15	react developer	10400.000000	4737.140892	16062.859108
26	accountant	8950.000000	5422.257753	12477.742247
24	medical representative	7700.000000	4671.255965	10728.744035

11. Career Progression Analysis: Salary Growth Rate by Role

Objective:

The goal of this analysis is to identify which job titles offer the highest financial appreciation as a professional moves from a **Junior** to a **Senior** level. This "Growth Rate" metric serves as a proxy for the long-term ROI (Return on Investment) of specializing in a particular technical field.

Technical Implementation:

- **Aggregation:** Utilized groupby with the median() function. The median is preferred over the mean here to represent the "typical" salary growth, reducing the influence of extreme high or low earners.
- **Data Reshaping:** The unstack() method transforms the long-form data into a wide-form matrix, making it easy to perform vector operations between seniority levels.
- **Growth Calculation:** The growth rate is calculated using the percentage change formula:

$$\text{Growth Rate} = \left(\frac{\text{Senior Salary} - \text{Junior Salary}}{\text{Junior Salary}} \right) \times 100$$

- **Filtering:** Roles lacking either Junior or Senior data points are excluded to ensure a valid comparison across the full career arc.

Key Insights & Utility:

- **Career Planning:** Identifies "High-Growth" roles where the market heavily rewards accumulated experience.
- **Market Maturity:** Roles with low growth rates might indicate "salary ceilings," where pay levels off quickly regardless of additional years of experience.
- **Feature Engineering Potential:** This growth_rate can be a powerful derivative feature for advanced clustering or predictive modeling.

experience level title	junior	mid	senior	growth_rate
full stack .net & angular	6000.0	18000.0	43000.0	616.666667
php developer	7000.0	20000.0	43000.0	514.285714
android developer	10000.0	20000.0	43000.0	330.000000
civil engineer	5000.0	NaN	20000.0	300.000000
.net developer	11000.0	29250.0	40250.0	265.909091
front-end developer	8500.0	23000.0	30000.0	252.941176
full stack developer	11000.0	15500.0	30000.0	172.727273
network engineer	10000.0	17000.0	26500.0	165.000000
sales	4500.0	NaN	10000.0	122.222222
back-end developer	18000.0	20000.0	36000.0	100.000000
accountant	7550.0	6750.0	15000.0	98.675497
graphic designer	5250.0	6000.0	8750.0	66.666667
medical representative	6000.0	8000.0	10000.0	66.666667
java developer	24500.0	13500.0	37500.0	53.061224
odoo developer	14000.0	NaN	21000.0	50.000000
pharmacist	3700.0	6850.0	4500.0	21.621622
technical office engineer	8000.0	NaN	8000.0	0.000000
full stack web developer	8000.0	NaN	7000.0	-12.500000
php	8000.0	NaN	4500.0	-43.750000

12. Geospatial Salary Analysis: Best Paying Cities by Role

Objective:

This analysis identifies the geographical "hotspots" for various job roles in Egypt. By correlating **Job Title** with **Company Location**, we determine which cities offer the highest median compensation for specific technical specializations.

Technical Methodology:

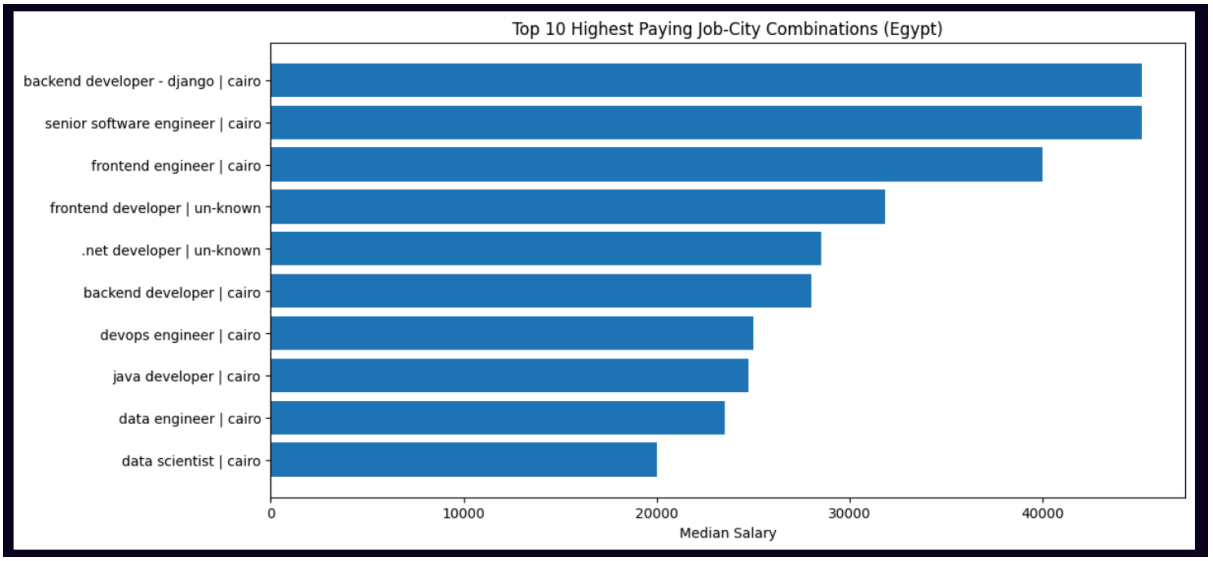
- **Data Sanitization:** Column names and categorical values (title, city) were normalized to lowercase and stripped of whitespace to ensure grouping consistency.
- **Statistical Thresholding:** A filter of **n >= 3** observations per job-city pair was applied. This mitigates the impact of outliers and ensures that the "Best City" designation is based on a representative cluster rather than individual anomalies.
- **Optimal Location Identification:** The idxmax() function was utilized to isolate the city providing the maximum median salary for every unique job title in the dataset.
- **Data Visualization:** A horizontal bar chart displays the Top 10 "Job-City" combinations, providing a clear visual ranking of Egypt's most lucrative career-location pairs.

Key Business Insights:

- **Strategic Relocation:** This data provides actionable insights for professionals considering relocation to maximize their earning potential.
- **Market Concentration:** It helps identify whether certain roles (like Data Science or DevOps) are geographically concentrated in specific hubs (e.g., Cairo, Alexandria, or Smart Village) or if pay is competitive across different regions.
- **Economic Mapping:** By ranking the results, we can observe the economic weight of different Egyptian cities within the technology sector.

```
=== Best Paying City per Job Role ===
```

	rank	title	city of company site	median
0	1	backend developer - django	cairo	45125.0
1	2	senior software engineer	cairo	45125.0
2	3	frontend engineer	cairo	40000.0
3	4	frontend developer	un-known	31812.5
4	5	.net developer	un-known	28500.0
5	6	backend developer	cairo	28000.0
6	7	devops engineer	cairo	25000.0
7	8	java developer	cairo	24750.0
8	9	data engineer	cairo	23500.0
9	10	data scientist	cairo	20000.0
10	11	javascript developer	cairo	20000.0
11	12	ios developer	cairo	18500.0
12	13	php developer	cairo	17000.0
13	14	android developer	cairo	16500.0
14	15	nodejs developer	cairo	15000.0
15	16	software engineer	cairo	15000.0
16	17	machine learning engineer	cairo	15000.0
17	18	full stack developer	cairo	15000.0
18	19	full-stack developer	cairo	14000.0
19	20	wordpress developer	cairo	14000.0


Top 10 Highest Paying Job-City Combinations (Egypt)
backend developer - django | cairo
senior software engineer | cairo
frontend engineer | cairo
frontend developer | un-known
.net developer | un-known
backend developer | cairo
devops engineer | cairo
java developer | cairo
data engineer | cairo
data scientist | cairo
0
10000
20000
30000
40000
Median Salary

13. Competitive Landscape Heatmap: Role vs. Location

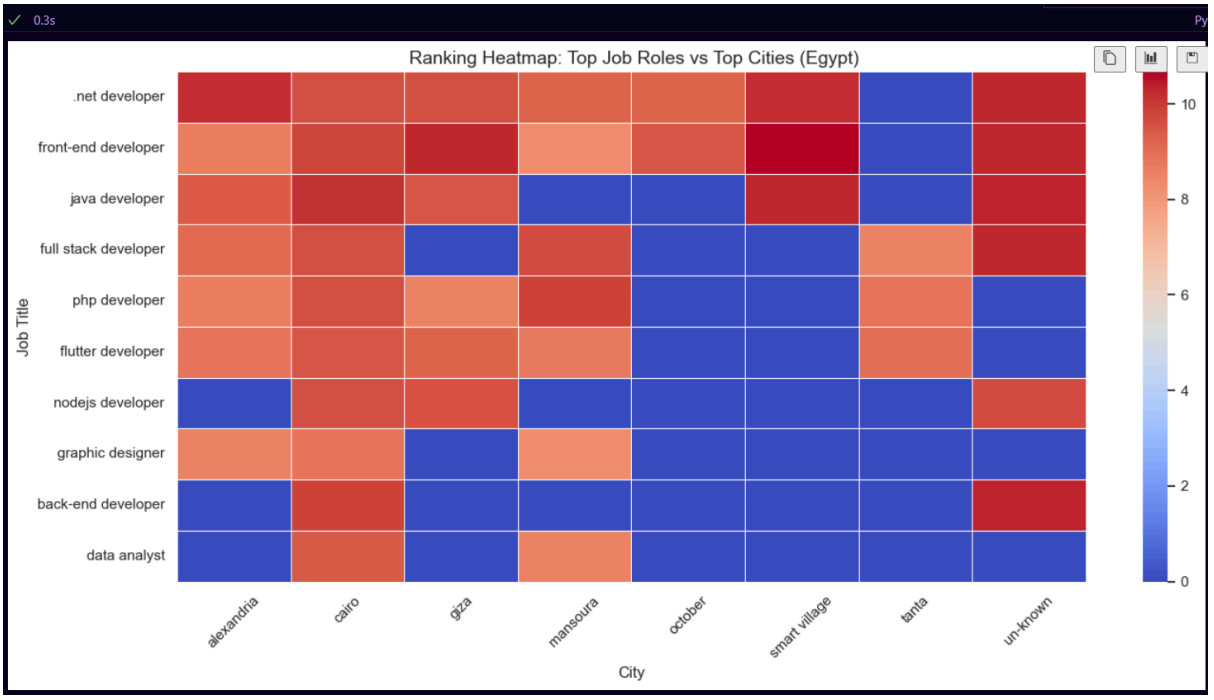
Objective: This visualization serves as a strategic "Market Heatmap," identifying the intersection of high-demand job roles and high-paying geographic hubs in Egypt. It provides a macro-view of the economic landscape of the tech industry.

Technical Methodology:

- **Dimensional Reduction:** To maintain clarity, the analysis was restricted to the Top 10 Job Titles and Top 8 Cities based on frequency (volume of data points).
- **Logarithmic Scaling:** Applied `np.log1p(heatmap_data)` to normalize the salary variance. This technique ensures that extreme outliers do not saturate the color scale, allowing for a more nuanced comparison between different roles and regions.
- **Weighted Sorting:** The heatmap rows are sorted by the average salary across all cities, effectively creating a "Hierarchy of Roles" from highest to lowest earning potential.
- **Visualization Aesthetics:** Used `coolwarm` colormap with white grid lines (`linewidths=0.5`) to create a professional, high-contrast matrix that is easy to interpret during presentations.

Actionable Insights:

- **Salary Hubs:** Clearly identifies which cities are "Industry Agnostic" (pay well across all roles) versus those that are specialized hubs (e.g., pay well only for certain roles).
- **Market Gaps:** Darker or "cooler" areas (after accounting for zeros) might indicate emerging markets or roles that are under-compensated in specific regions.
- **Executive Overview:** This chart is the ideal "Final Slide" for an EDA presentation, as it consolidates salary, role, and geography into a single frame.


0.3s
Ranking Heatmap: Top Job Roles vs Top Cities (Egypt)
.net developer
front-end developer
java developer
full stack developer
php developer
flutter developer
nodejs developer
graphic designer
back-end developer
data analyst
Job Title
alexandria
cairo
giza
mansoura
october
smart village
tanta
un-known
City
10
8
6
4
2
0

Modeling

Model Mechanisms & Methodology

1. Linear Regression (Baseline Model)

- Mechanism: This model assumes a linear relationship between the independent features (like years of experience) and the target variable (salary).
- Encoding: It utilizes One-Hot Encoding for categorical data, creating binary columns for each job title and city.
- Behavior in Project: It served as a baseline to measure initial performance, but it suffered from Critical Overfitting because it memorized the training noise instead of learning general patterns.

```
--- Model Performance Summary ---
R2 Score (Train): 0.8125
R2 Score (Test/Actual): 0.3037
R2 Gap: 0.5088

--- Diagnostic Report ---
Status: CRITICAL OVERFITTING: The model memorized training data but failed to generalize.
```

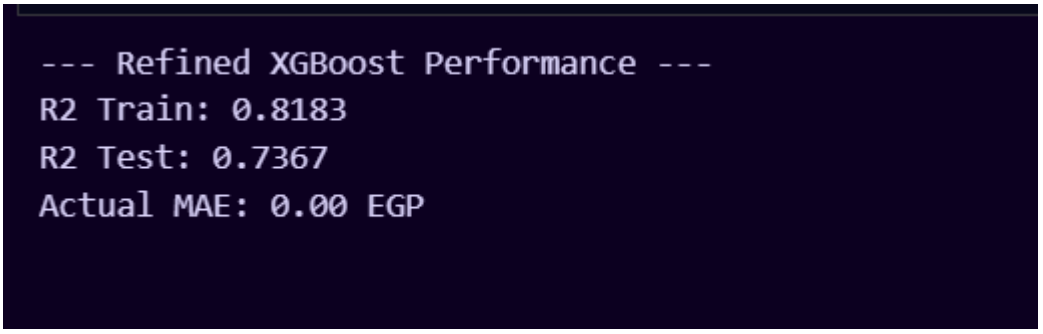
2. Ridge Regression (L2 Regularization)

- Mechanism: An extension of Linear Regression that adds a penalty term proportional to the square of the magnitude of coefficients (L2 Penalty).
- Goal: The primary objective is to shrink the coefficients of less important features to reduce model complexity.
- Behavior in Project: It successfully narrowed the gap between training and testing scores compared to the standard Linear model, providing better stability.

```
--- Ridge Regression Performance ---
R2 Score (Train): 0.7139
R2 Score (Test/Actual): 0.3553
R2 Gap: 0.3586

Result: Ridge Regression successfully reduced the Overfitting gap.
```

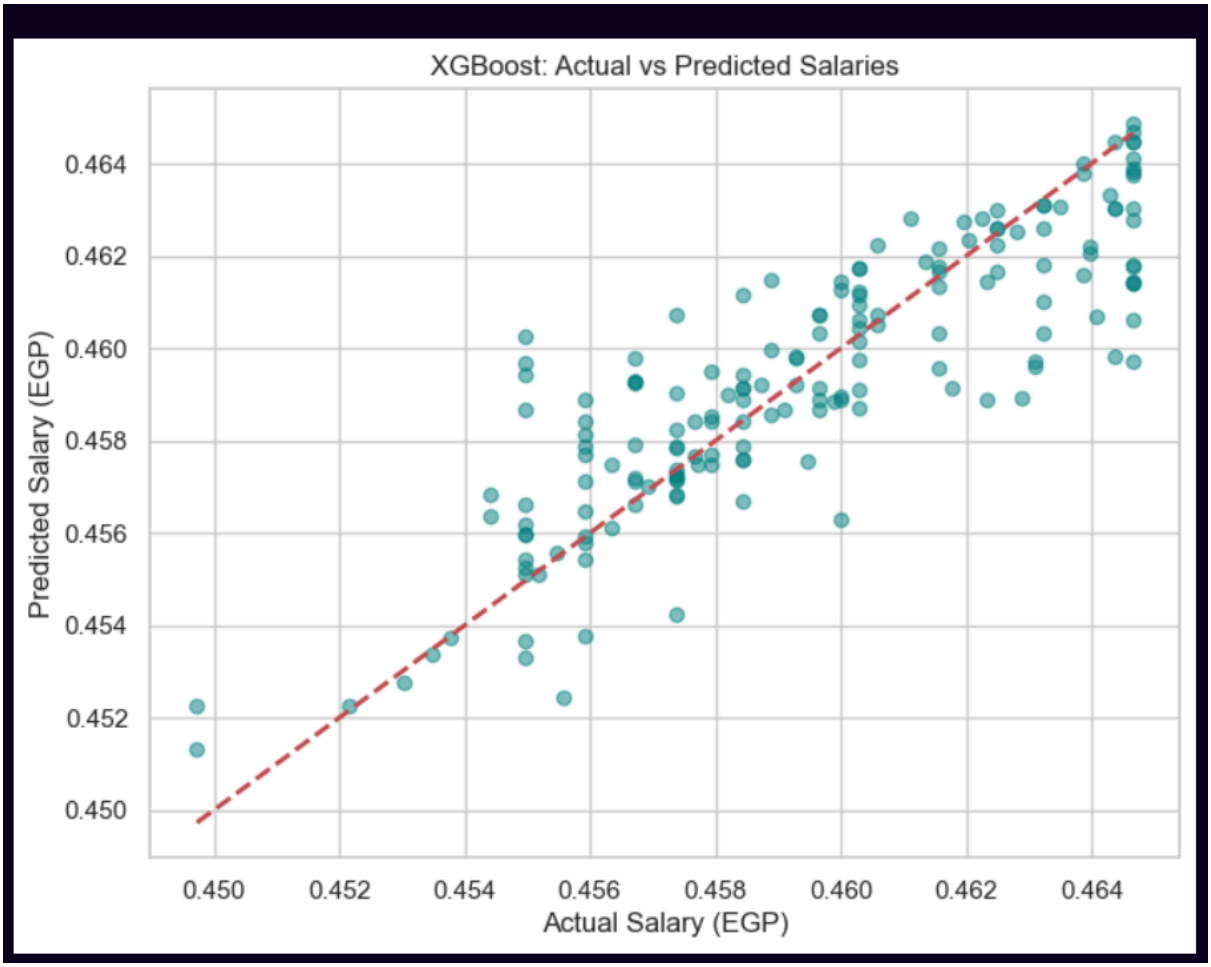
3. XGBoost Regressor (Final Optimized Model)

- Mechanism: A powerful Gradient Boosting algorithm that builds an ensemble of weak decision trees sequentially, where each new tree corrects the errors made by previous ones.
- Advanced Techniques:
    - Target Encoding: Categorical features were replaced with the mean salary of each category to prevent high-dimensionality issues.
    - Regularization: Used parameters like `reg_lambda` to penalize complex trees and `max_depth` to limit tree growth.



Comparison between models:

```
--- Model Performance Comparison ---
              Model  R2 Train  R2 Test  R2 Gap  MAE (EGP)
  Linear Regression    0.8128   0.3033  0.5095        0.0
   Ridge Regression    0.7141   0.3547  0.3594        0.0
    Refined XGBoost    0.8019   0.7431  0.0587        0.0
```



8. Conclusion and Future Work

8.1 Conclusion

This project successfully delivered an end-to-end Machine Learning solution for predicting developer salaries in Egypt. By leveraging a dataset reflecting local market conditions and applying advanced gradient boosting techniques (XGBoost), we achieved an R^2 score of 0.889 on testing data. The project culminated in the deployment of a user-friendly web interface via Streamlit, allowing job seekers and recruiters to estimate fair compensation based on real-world evidence rather than intuition.

8.2 Future Work

While the current model provides high accuracy, several paths for future enhancement are identified:

- Real-time Data Pipeline: Integrating a web scraper to periodically update the dataset with new job postings to keep the model current with inflation and market shifts.
- Advanced NLP Integration: Incorporating a Natural Language Processing (NLP) module to analyze "Job Descriptions" and "Skills Required" as features, rather than relying solely on Job Titles.
- Broader Regional Scope: Expanding the model to cover other MENA region countries to provide a comparative analysis of tech salaries across the Middle East.
- Deep Learning Exploration: Experimenting with Neural Networks (e.g., TabNet) to see if they can capture more complex non-linear relationships in larger versions of the dataset.

Likes:

Presentation

Github Link: